

NAME: Atchaya Vharsne S

REG. NO.: 192524185

COURSE CODE: CSA0401

COURSE NAME: OPERATING SYSTEMS

CO5 AT3 MOCK INTERVIEW

1. Linux File System Analysis and Long-Term Solution

The Linux file system can be analysed using commands such as:

```
df -h  
  
du -sh /*  
  
lsblk  
  
mount
```

- df -h checks disk space usage.
- du identifies directories consuming more space.
- lsblk displays disk partitions.
- mount shows mounted file systems.

Long-term solution:

- Implement disk usage monitoring.
- Configure log rotation using logrotate.
- Set disk quotas for users if required.
- Regularly clean temporary and unnecessary files.
- Use automated alerts when disk usage reaches a threshold.

2. Extract Failed Login Attempts from a 10 GB Log File

To efficiently find failed login attempts within a specific one-minute time range, use `grep` with the timestamp.

Example:

```
grep "Aug 27 10:30" /var/log/auth.log | grep "Failed"
```

A more efficient approach for large files is:

```
grep -F "Aug 27 10:30" large_log.txt | grep -F "Failed"
```

Performance optimization strategies:

- Use `grep` instead of opening the entire file in an editor.
- Use `-F` for faster fixed-string searching.
- Compress and archive old logs.
- Use `awk` if more advanced filtering is required.

Example:

```
awk '/Aug 27 10:30/ && /Failed/' large_log.txt
```

This extracts only the required failed login records without manually processing the entire file.

3. Process Consuming 95% CPU

First, identify the process using:

```
top  
  
or:  
  
htop
```

To find high CPU processes:

```
ps aux --sort=-%cpu | head
```

After identifying the PID, evaluate the process to determine whether it is a critical system service.

If it is safe to manage:

```
renice +10 -p PID
```

This reduces its CPU priority.

If necessary:

```
kill PID
```

For stronger control:

```
kill -9 PID
```

Conclusion: Identify the process, check its importance, reduce its priority or limit resources, and terminate it only if necessary to avoid disrupting critical services.

4. Type 1 vs Type 2 Hypervisor

I would generally recommend a Type 1 hypervisor for server or enterprise environments.

Type 1 Hypervisor

- Runs directly on physical hardware.
- Provides better performance.
- Has lower overhead.
- Supports better scalability.
- Suitable for servers and data centres.

Type 2 Hypervisor

- Runs on top of a host operating system.
- Easier and cheaper for personal use and testing.
- Has more overhead.

Conclusion:

For high performance and scalability, Type 1 is better. For low-cost learning, testing, or small-scale use, Type 2 is suitable.

5. Resource Allocation Strategy for a 16 GB RAM Server with Multiple VMs

The 16 GB RAM should not be allocated completely to virtual machines because the host operating system also requires memory.

Example allocation:

- Host OS: 2 GB
- VM 1: 4 GB
- VM 2: 3 GB
- VM 3: 3 GB
- VM 4: 2 GB
- Reserved memory: 2 GB

The allocation should depend on the workload of each VM.

Efficient strategy:

- Allocate more memory to high-priority VMs.
- Use dynamic memory allocation or ballooning if supported.
- Monitor RAM and CPU utilization regularly.
- Avoid excessive over-allocation.
- Set CPU and memory limits for each VM.

This ensures balanced resource usage and prevents one VM from affecting the others.

6. Resource Isolation for Multiple Applications

To prevent one application from negatively affecting others, use resource isolation through Linux cgroups.

Example strategy:

- Create separate cgroups for each application.
- Set CPU limits.
- Set memory limits.
- Assign each application's process to its respective cgroup.

Example:

```
systemd-run --scope -p CPUQuota=30% -p MemoryMax=2G application
```

This limits the application to 30% CPU and 2 GB RAM.

Other useful mechanisms include:

- `nice` and `renice` for CPU priority.
- `ulimit` for process resource limits.
- Docker or containers for stronger application isolation.